第 4 周周报

一、本周工作

1. 架构调整：去掉反馈节点

本周对第 3 周的 LangGraph 拓扑做了一次结构性简化，移除了 `feedback` 节点，理由有两点：

原因	说明
反馈节点需额外调用一次大模型	每轮多一次 VL 推理，端到端延迟明显上升，实测速度较慢
反馈节点与感知节点截图重复	feedback 为看执行后画面需再截图，而下一轮 perception 本来就会重新截图，两次截图重复且浪费

调整后的思路：把"任务是否完成"的判断合并进 planner 节点——planner 在每轮决策时直接基于当前截图判断任务是否已经完成，若完成则输出终止信息，不再单独设一个反思/反馈环节。如果后续测试中发现纠错或反思能力不足，会及时补充相应环节。

2. 其他改动

2.1 在提示词内部补充可选动作列表和选择依据

问题：此前提示词里没有可选动作列表，动作选择单纯由 `output_schema`的字段约束。结构化输出只保证"输出格式合法"，但不告诉模型"什么场景该选哪个动作"，导致模型在动作选择上缺乏依据。

修改：在系统提示词中加入可选动作列表，逐个列出动作及其适用场景。效果是模型选择动作时有明确依据，同时减少结构化输出校验失败。

2.2 OutputSchema：精简动作集

问题：原动作集过于原子，包含"保持鼠标一直按着""只抬起鼠标"这类底层动作，动作空间大、语义模糊，模型选择困难。

修改：精简可选动作集，去掉过于原子的动作，保留语义完整、可独立成步的动作，使动作集更简明，模型选择更方便。

3. 命令行交互开发

基于 Typer 实现 `src/cli.py`，核心逻辑：

```
import typer
from agent import GUIAgent

app = typer.Typer()

@app.command()
def run(goal: str):
```

```
    agent = GUIAgent() # 初始化agent
    agent.run_stream(goal) # 流式运行agent


if __name__ == "__main__":
    app()
```

---

4. 五个基础任务测试

每个任务记录包含四部分：

- 命令行命令
- 真机运行录屏
- 打印日志
- 分析问题

任务 1：打开浏览器

- 命令：

```
python src/cli.py "open the web browser"
```

- 录屏：演示视频
- 打印日志：点击链接查看完整日志

```
[step 1] action={'type': double_click, 'x': 52, 'y': 408}
[step 2] action={'type': left_click, 'x': 1120, 'y': 304}
[step 3] action={'type': finish}  finished=True
```

- 问题分析：
  - 成功完成任务。

任务 2：在浏览器里搜索指定内容

- 命令：

```
python src/cli.py "search for 'LangGraph tutorial' in the browser"
```

- 录屏：演示视频
- 打印日志：点击链接查看完整日志

```
[step 1] action={'type': left_click, 'x': 600, 'y': 565}
[step 2] action={'type': left_click, 'x': 600, 'y': 565}
[step 3] action={'type': left_click, 'x': 600, 'y': 565}
[step 4] action={'type': type_text, 'text': 'LangGraph tutorial'}
[step 5] action={'type': finish}  finished=True
```

- 问题分析：
  - 成功，但 step 1~3 连续三次 `left_click` 落在同一坐标 `(600,565)`，属于重复动作。说明"点击搜索框"这一步被重复执行。

任务 3：打开指定文件

- 命令：

```
python src/cli.py "open the file named test_function in the file explorer"
```

- 录屏：演示视频
- 打印日志：点击链接查看完整日志

```
[step 1] action={'type': double_click, 'x': 405, 'y': 591}
[step 2] action={'type': left_click, 'x': 1092, 'y': 1062}
[step 3] action={'type': left_click, 'x': 1092, 'y': 1062}
[step 4] action={'type': left_click, 'x': 1092, 'y': 1062}
[step 5] action={'type': left_click, 'x': 1092, 'y': 1062}
```

- 问题分析：
  - 根据录屏可以看到任务完成，但是 agent 未检测到任务完成，从第三步开始进入死循环。

任务 4：发送消息

- 命令（按日志实际执行）：

```
python src/cli.py "type 'hello' into the chat box and click 发送"
```

- 录屏：演示视频
- 打印日志：点击链接查看完整日志

```
[step 1] action={'type': type_text, 'text': 'hello'}
[step 2] action={'type': left_click, 'x': 1872, 'y': 988}
[step 3] action={'type': left_click, 'x': 1872, 'y': 988}
[step 4] action={'type': left_click, 'x': 1872, 'y': 988}
```

```
[step 5] action={'type': left_click, 'x': 1872, 'y': 988}
[step 6] action={'type': left_click, 'x': 1872, 'y': 988}
```

- 问题分析：
  - 根据录屏可以看到任务完成，但是出现严重重复动作的问题导致任务未闭环。step 1 成功输入 `hello`，step 2~6 连续五次 `left_click` 落在同一坐标 `(1872,988)`（发送按钮），属于典型的重复动作死循环，与任务三出现同样的问题。

任务 5：关闭浏览器

- 命令（按日志实际执行）：

```
python src/cli.py "close the Firefox browser"
```

- 录屏：演示视频
- 打印日志：点击链接查看完整日志

```
[step 1] action={'type': left_click, 'x': 1903, 'y': 25}
[step 2] action={'type': left_click, 'x': 40, 'y': 412}
[step 3] action={'type': left_click, 'x': 1166, 'y': 1061}
[step 4] action={'type': finish}  finished=True
```

- 问题分析：
  - 日志标记 finished=True，但过程有问题。step 2 动作方向错误：浏览器已关闭，agent 却点击桌面 Firefox 图标 `(40,412)`——thought 写"点击它会打开浏览器，为关闭需先确保打开"，与 goal 相反，属于误判。

任务测试汇总

五个任务测下来，主要两个问题：

- 最终任务完成了却判断不出来
- 多步任务会卡在某一步反复输出相同动作。

这两个其实是同一个问题——都是"某一步操作到底有没有完成"判断不准，前者是最终任务粒度，后者是子步骤粒度。

二、遇到的问题与解决

问题一：本地推理速度过慢。 目前用本地推理，真机录屏中没有做任何加速，单步操作平均要 1 分钟左右，整体效率很低。

问题二：推理模型能力偏弱。 用的是 Qwen2.5-VL-7B，真机是 4050、6GB 显存，模型规模和硬件都受限，影响决策质量。

问题三：harness 设计能力不足。 调整框架更多靠感觉——有时调提示词，有时调输出结构，有时改记忆设计，缺少系统性的方法论来定位"到底该改哪一层"。

问题四：反馈节点需要重新设计。 本周为了提速去掉了独立 feedback 节点，但测试暴露出"完成判断不准、重复动作"是主要问题，而这两点恰恰需要反馈机制来解决。因此下一步不是简单恢复原节点，而是重新设计反馈节点，让它承担"确认动作是否生效、判断子步骤与最终任务是否完成"的职责。

后续方向： 把推理迁到云端并选用更大的模型；在 harness 设计上沉淀可复用的调试方法；重新设计反馈节点，从机制上解决完成判断与重复动作问题。

三、下周计划

第 5 周：多模态大模型 LoRA 微调与能力提升

模块	任务
训练集构建	基于预处理后的公开 GUI 数据集，构建微调训练集与验证集
LoRA 微调	使用 PEFT 库实现对开源多模态模型的 LoRA 微调
效果对比	对比微调前后模型在 GUI 任务理解与动作生成上的效果
提示词优化	优化提示词工程，提升模型的任务执行准确率

数据来源

1. Hugging Face Transformers、PEFT、Accelerate 库
2. 前序处理的公开数据集
3. 本地 GPU 或 Google Colab GPU 资源

下周交付物：微调后的模型权重 + 微调效果对比分析报告